Split-Sync App

Shared Expense Management System

Project Report

Started 24/10/2025

Student Name: Emmanuel Mobolaji Abolade

Student ID: C00288657

Institution: South East technological University, Carlow

Programme: BSc Software Development

Supervisor: Joseph Kehoe

Submitted: 17/04/2026

Acknowledgements

I would like to express my sincere gratitude to my project supervisor, Joseph Kehoe, for his guidance, feedback, and support throughout the development of this Final Year Project. His expertise and advice were invaluable in shaping the direction and quality of this work.

I would like to thank the faculty and staff of the Department of Computing and Mathematics at South East Technological University (SETU) Carlow for providing the academic foundation that made this project possible.

I am grateful to the eighteen participants who took part in the user evaluation study. Their honest feedback and suggestions directly influenced the final design and functionality of Split-Sync.

I would also like to acknowledge the open-source community whose libraries, tools, and documentation made the technical implementation of this project possible, including the React, Firebase, Tailwind CSS, and Lucide React communities.

Finally, I thank my family and friends for their patience, encouragement, and support throughout this academic year.

TABLE OF CONTENTS

Acknowledgements **1**
List of Tables **5**
1. Introduction **7**
2. Project Overview and Scope **8**
2.1 Application Summary 8
2.2 Core Features Scoped 8
2.3 Scope Boundaries 9
3. Benefits of Split-Sync **10**
3.1 Financial Clarity in Group Settings 10
3.2 Flexible and Fair Expense Splitting 10
3.3 Integrated Payment Methods Relevant to European Users 11
3.4 Privacy by Design 11
3.5 Progressive Web App Advantages 11
3.6 Academic and Professional Value 12
4. Limitations of the Project **13**
4.1 OCR Receipt Scanning 13
4.2 Test Coverage Distribution 13
4.3 No End-to-End Testing 14
4.4 SEPA Payment Processing 14
4.5 Single Developer Constraint 14
4.6 Limited User Testing 15
5. Development Journey and Stage Achievements **16**
5.1 Stage One: Research, Planning, and Initial Setup 16
5.2 Stage Two: Core Features - Groups and Expenses 16
5.3 Stage Three: Payment Integration and Notifications 17
5.4 Stage Four: Testing Infrastructure 18
5.5 Stage Five: UX Refinement and Final Features 19
5. 6. Project Delivery Checklist 20
6. Problems Encountered and How They Were Resolved **26**
6.1 Firebase Security Rules Blocking Legitimate Reads 26
6.2 React Testing Library and Firebase Mock Complexity 27
6.3 Balance Calculation Edge Cases 27
6.4 OCR Receipt Scanning Accuracy 28
6.5 Dark Mode Inconsistency Across Components 28
6.6 Session Timeout and Navigation Race Conditions 29
7. What Was Not Achieved and Why **30**
7.1 Full OCR Receipt Scanning 30

7.2 End-to-End Testing..... 30
7.3 Export Functionality (PDF and CSV)..... 31
7.4 Budget Tracking and Alerts..... 31
7.5 Multi-Language Support..... 31
8. What I Would Do Differently if Starting Again..... 32
8.1 Write Tests From Day One..... 32
8.2 Invest More Time in Data Modelling Upfront..... 32
8.3 Use TypeScript..... 33
8.4 Scope the OCR Feature More Conservatively..... 33
8.5 Set Up CI/CD Earlier..... 33
8.6 Engage Supervisor More Regularly..... 34
9. Mobile vs Desktop comparison..... 35
Table 3: Mobile vs Desktop Legend..... 36
9.1 Core Expense Management..... 36
9.2 Balance Tracking and Settlement..... 39
9.3 Payment Integrations..... 40
9.4 Authentication and Security..... 42
9.5 Group Management..... 43
9.6 Receipt Scanning (OCR)..... 45
9.7 Notifications..... 46
9.8 User Interface and Experience..... 48
9.9 Spending Analytics..... 50
9.10 Progressive Web App (PWA) Capabilities..... 51
9.11 Settings and Profile Management..... 52
10. User Evaluation..... 56
10.1 Methodology..... 56
10.2 Participant Overview..... 56
10.3 Overall Experience..... 56
10.4 Feature Ratings..... 57
10.5 Interface and Performance..... 58
10.6 Qualitative Findings..... 58
10.7 Comparative Analysis..... 59
10.8 Net Promoter Score..... 60
10.9 Observational and Informal User Testing..... 61
10.9. 1 Cybersecurity Peer Review..... 61
10.9.2 General App Testers..... 61
10.10 Reflection and Response to Feedback..... 62
11. Conclusion..... 63
Glossary..... 64
Appendix..... 70

Appendix A: Project Links and Deployed Artefacts..... 70
Appendix B: Test Coverage Summary Table..... 71
Appendix C: User Evaluation Questionnaire Structure..... 71
Appendix D: Firebase Security Rules (Simplified Extract)..... 73
Appendix E: IBAN Validation Algorithm..... 73
Appendix F: Settlement Algorithm Description..... 74
Bibliography..... 75

List of Tables

All tables presented in this report are listed below with corresponding page references.

Table Number	Table Title	Page
Table 1	Project Delivery checklist	20
Table 2	Project Delivery checklist Summary	24
Table 3	Mobile vs Desktop Legend	35
Table 4	Core Expense Management	36
Table 5	Balance Tracking and Settlement	39
Table 6	Payment Integrations Feature Comparison	40
Table 7	Authentication and Security Feature Comparison	42
Table 8	Group Management Feature Comparison	43
Table 9	Receipt Scanning (OCR) Feature Comparison	45
Table 10	Notifications Feature Comparison	46

Table 11	User Interface and Experience Feature Comparison	48
Table 12	Spending Analytics Feature Comparison	50
Table 13	Progressive Web App (PWA) Capabilities Feature Comparison	51
Table 14	Settings and Profile Management Feature Comparison	52
Table 15	User Evaluation Feature Ratings by Participants	57
Table 16	Net Promoter Score Classification	60

1. Introduction

This report presents a detailed account of my Final Year Project (FYP) for the BSc in Software Development at South East Technological University (SETU) Carlow. The project, named Split-Sync, is a Progressive Web Application (PWA) designed to simplify group expense management, bill splitting, and payment settlement. It was developed over the course of the 2025/2026 academic year under the supervision of Joseph Kehoe.

The idea for Split-Sync came from a frustration I personally experienced when living with housemates and trying to fairly split shared costs such as rent, groceries, and utility bills. Existing solutions like Splitwise, Settle Up and Tikkie while functional, felt clunky on mobile and lacked seamless payment integrations relevant to the Irish and European market, such as SEPA bank transfers and Revolut. I wanted to build something modern, fast, and genuinely useful.

This report documents the entire journey: from the initial concept and scope definition, through the development milestones and technical challenges, to the final testing results and honest reflections on what I achieved, what fell short, and what I would do differently.

2. Project Overview and Scope

2.1 Application Summary

Split-Sync is a full-stack Progressive Web Application built with React 18, Firebase, Tailwind CSS, and Zustand for state management. It enables groups of users to record shared expenses, automatically calculate who owes what, and settle balances through integrated payment methods. The application is deployed on Firebase Hosting and is accessible at https://splitsync-dev.web.app .

2.2 Core Features Scoped

- User authentication via email/password and Google OAuth 2.0
- Group creation and member management with invite systems
- Expense entry with four splitting methods: equal, percentage, exact amounts, and shares
- Real-time balance calculations and smart settlement suggestions
- Payment integration supporting PayPal, Revolut, and SEPA bank transfers
- OCR-powered receipt scanning via Google Cloud Vision API
- Privacy-preserving contact synchronisation using SHA-256 phone number hashing
- Biometric authentication via the WebAuthn API
- Real-time notifications for expenses, payments, and group invitations
- Dark mode support with system preference detection
- Progressive Web App capabilities including offline support and installability

- Comprehensive testing infrastructure with a minimum 50% code coverage target

2.3 Scope Boundaries

From the outset, certain features were deliberately placed outside the project scope to keep delivery realistic within the academic timeline. These included a native mobile application, a full banking API integration, multi-language internationalisation, and a group messaging system. These are acknowledged in Section 7 as planned future enhancements.

3. Benefits of Split-Sync

Throughout the design and development of this project, it became clear that Split-Sync addresses several genuine pain points that existing tools do not fully solve, particularly for users in the Irish and European context.

3.1 Financial Clarity in Group Settings

One of the most common sources of tension in shared living arrangements or group travel is the ambiguity around who owes what. Split-Sync eliminates this by providing real-time balance calculations that are always accurate and visible. The dashboard gives every group member immediate clarity on their financial standing without needing to trace back through old messages or spreadsheets.

3.2 Flexible and Fair Expense Splitting

Unlike basic 50/50 tools, Split-Sync supports four distinct splitting methods. This matters in practice: not every shared expense should be split equally. For example, if two people share a large apartment and one person has a smaller room, a percentage or shares-based split is more appropriate. Having these options built into the application makes it genuinely fair rather than just mathematically convenient.

3.3 Integrated Payment Methods Relevant to European Users

Most competing applications are built primarily for the American market and focus on Venmo or PayPal. Split-Sync was designed with European users in mind, specifically incorporating SEPA bank transfer support with IBAN validation, and Revolut deep links, both of which are far more common payment tools in Ireland and across the EU. This represents a meaningful market gap that the application addresses.

3.4 Privacy by Design

The contact synchronisation feature allows users to find friends who are already on the platform without ever uploading their contacts to a server. Phone numbers are hashed locally using SHA-256 before any comparison occurs. This approach aligns with GDPR principles and represents a technically thoughtful solution to a common privacy concern with social applications.

3.5 Progressive Web App Advantages

By building Split-Sync as a PWA, the application can be installed on any device without requiring App Store approval or platform-specific development. It supports offline functionality through service workers, making it usable even without an internet connection. This lowers the barrier to adoption significantly and demonstrates a modern approach to cross-platform deployment.

3.6 Academic and Professional Value

Beyond its practical utility, this project served as a vehicle for developing skills across the full development stack: React component architecture, Firebase backend configuration, state management, security rule design, API integration, and testing. The breadth of technologies employed reflects the complexity expected at final year level and provides a portfolio piece demonstrating real-world application development capability.

4. Limitations of the Project

Being transparent about limitations is as important as highlighting achievements. Several constraints, both technical and time-related, meant that certain aspects of Split-Sync did not reach the level of completion I had originally envisioned.

4.1 OCR Receipt Scanning

Receipt scanning was one of the features I was most excited about. The integration with the Google Cloud Vision API was implemented in the `ReceiptScanner` component, and it is capable of extracting merchant names, totals, and transaction dates from uploaded images. However, the accuracy is inconsistent with low-resolution or poorly lit images, and the feature did not reach a production-ready state. It remains partially functional and is not exposed in the main application flow for regular users. This is arguably the biggest gap between what was intended and what was delivered.

4.2 Test Coverage Distribution

While the overall test coverage of 84.37% lines across 1,397 tests in 21 test suites, with zero failing tests at the point of submission met the academic milestone, the coverage is uneven. Critical UI components such as the `AddExpenseModal`, `SettleUpModal`, `SepaPaymentModal`, and several others have zero test coverage, as they proved extremely challenging to test due to their deep dependency on Firebase Firestore real-time listeners and complex modal state. The high utility-function coverage compensates statistically, but it does mean that some of the most user-facing code is untested.

4.3 No End-to-End Testing

The testing infrastructure relies entirely on unit tests and React Testing Library component tests. End-to-end testing using a tool like Cypress or Playwright was planned but not implemented within the project timeline. This means that the full user journey, from registration through expense entry to settlement, has not been tested in an automated way against a real browser environment.

4.4 SEPA Payment Processing

The SEPA integration generates bank transfer details and validates IBANs, but it does not actually initiate bank transfers. That would require integration with a licensed payment institution or a banking API such as TrueLayer or Stripe's Financial Connections product, which are beyond the scope and budget of an academic project. Users are presented with the transfer details and must execute the payment themselves through their own banking application. However, it was implemented as a test demo which worked as it should if a business account was used to set up in the developer mode.

4.5 Single Developer Constraint

Being the sole developer on a project of this scope was both a learning opportunity and a practical constraint. There was no code review process, no pair programming, and no separate QA resource. Decisions that might have been caught early in a team environment sometimes became technical debt that required revisiting later. The trade-off was that I developed a much

deeper understanding of every part of the system, but the pace of delivery was slower than it might have been in a team.

4.6 Limited User Testing

Formal user testing was conducted on a small scale with a limited number of participants, primarily fellow students and friends. While feedback was collected and used to improve the interface, a larger and more diverse sample would have provided more actionable insights. Real-world usage patterns, particularly around the expense splitting flows, may reveal UX issues that were not surfaced during informal testing.

5. Development Journey and Stage Achievements

The project was developed iteratively over four months. What follows is an honest account of what was achieved at each stage and the reality of how each phase actually progressed.

5.1 Stage One: Research, Planning, and Initial Setup

The project began with a research phase during which I reviewed existing expense splitting applications, read academic papers on fintech PWA development, and explored the Firebase and React ecosystems. I produced an initial specification document and agreed on a feature set with my supervisor.

Setting up the development environment was more time-consuming than expected. Configuring Firebase Authentication, Firestore security rules, and the React project scaffold correctly from the start proved important because shortcuts taken here would have caused significant problems later. I spent approximately two weeks on this phase, and by the end I had a functional login system with both email/password and Google OAuth authentication, a basic project structure, and deployment to Firebase Hosting.

Key achievement at this stage: A deployed, authenticated shell application with routing established using React Router 7.

5.2 Stage Two: Core Features - Groups and Expenses

This was the most substantial development phase. Building the group management system required designing the Firestore data model carefully to support real-time updates, nested

member relationships, and role-based access. The expense entry system with four splitting methods was the most algorithmically complex piece of work in the entire project.

The balance calculator (balanceCalculator.js) became one of the most thoroughly tested parts of the application precisely because I was not confident the logic was correct until I had written tests for it. The function that calculates net balances across a group and produces optimal settlement paths is non-trivial, and debugging it through pure console logging would have been painful. Writing tests first for this module was genuinely the right decision.

I also built the invitation system at this stage, supporting QR code sharing, invite links, contact sync, and direct QuickAdd from previous connections. Each invitation method had its own technical challenges: QR code generation required a canvas-based component, invite links needed Firestore documents with expiry logic, and contact sync required implementing SHA-256 hashing in the browser before any data left the device.

Key achievement at this stage: Full expense management lifecycle from creation through balance calculation to settlement suggestion, with real-time Firestore sync across all group members.

5.3 Stage Three: Payment Integration and Notifications

Integrating payment methods was something I had been looking forward to but underestimated the complexity. PayPal and Revolut integrations work by generating deep links and redirect URLs that pre-populate payment amounts and recipient details. The SEPA integration required implementing IBAN validation logic from scratch, validating the checksum algorithm for over 30 European country formats.

The notifications system was built using Firestore real-time listeners with a Zustand store managing the notification state client-side. Designing the data model for notifications so that

they could be efficiently queried, marked as read, and cleaned up required several iterations. The first approach used a subcollection per user, which worked but became expensive in terms of Firestore reads. I refactored to a top-level notifications collection with userId indexing, which reduced read costs significantly.

The biometric authentication feature using the WebAuthn API was implemented for payment confirmation. This was technically interesting but proved difficult to test reliably because browser support varies and the testing environment does not expose biometric hardware.

Key achievement at this stage: A complete payment flow with three integrated payment methods, a functional real-time notification system, and biometric confirmation for payments.

5.4 Stage Four: Testing Infrastructure

Testing was something I honestly left later than I should have. At the point I began formally setting up the testing infrastructure, the coverage was at approximately 1.32%. The target of 50% felt significant at that stage.

The biggest challenge was mocking. Firebase Authentication, Firestore, and React Router all needed to be mocked at different levels. I created dedicated mock files for firebaseAuth.js, firebaseFirestore.js, and react-router-dom.js which were used as Jest manual mocks. Getting these right took considerable time and several failed test runs before I understood exactly what each component depended on.

I started with the utility functions because they had no UI dependencies and could be tested with straightforward input/output assertions. Once balanceCalculator.js, sepaPayment.js, paymentLinks.js, and the other utility modules were covered, the overall percentage climbed quickly. I then moved to stores, hooks, and finally page components.

By the February 26 milestone, I had reached 50.53% overall coverage with 817 tests across 29 test suites. By final submission, the coverage had grown to 84.37% line coverage across 1,397 passing tests in 52 suites, with all previously failing component tests resolved. Register.jsx and ForgotPassword.jsx achieved 100% coverage. The core utility layer reached over 92% statement coverage.

Key achievement at this stage: A robust testing infrastructure using React Testing Library and Jest, with a comprehensive mocking strategy for Firebase and React Router dependencies, and overall coverage exceeding the 50% milestone.

5.5 Stage Five: UX Refinement and Final Features

The final stage focused on polish, user experience improvements, and adding analytics. I replaced emoji usage throughout the application with Lucide React icons for a more consistent professional appearance. Dark mode support was audited across all components and gaps were filled. The analytics page was built using Recharts with animated charts showing spending trends, category breakdowns, and settlement histories.

The forgot password flow was rebuilt to handle Firebase error codes more gracefully, displaying meaningful messages rather than raw error strings. Session timeout functionality was implemented with a configurable warning modal that counts down before automatically signing the user out. The onboarding wizard was added to guide new users through the application for the first time.

Key achievement at this stage: A production-quality user interface with comprehensive dark mode support, animated data visualisation, improved error handling, and a smooth onboarding experience.

5. 6. Project Delivery Checklist

The following table provides a structured summary of every feature scoped in Section 2.2 against its implementation status at the point of submission. The purpose of this checklist is to provide a transparent account of delivery against the original project scope, supplementing the narrative in Section 5.

Feature	Status	Notes
Email and password authentication	Completed	Firebase Authentication with bcrypt hashing
Google OAuth login	Completed	One-tap Google sign-in on mobile and desktop
Biometric authentication (WebAuthn)	Completed	Fingerprint and Face ID via WebAuthn API
Session timeout and warning modal	Completed	Configurable inactivity period with countdown
Account deletion (GDPR)	Completed	Cascading deletion with soft-delete pattern

Group creation and management	Completed	Name, icon, currency, role-based membership
Member invitation via link	Completed	Shareable invite URL with expiry handling
Member invitation via QR code	Completed	Generated in-app with share modal
Quick Add previous collaborators	Completed	Connection graph built from shared group history
Contact sync (SHA-256 hashing)	Completed	Privacy-preserving phone number matching
Equal expense splitting	Completed	Rounding remainder assigned to payer
Percentage expense splitting	Completed	Auto-normalisation to 100% on mismatch
Exact amount expense splitting	Completed	Validation ensures splits match total
Shares-based expense splitting	Completed	Proportional allocation by share units

Real-time balance calculations	Completed	Greedy two-pointer settlement algorithm
Settlement suggestions	Completed	Minimum transaction optimisation
PayPal payment integration	Completed	PayPal.me deep link with pre-filled amount
Revolut payment integration	Completed	Revolut.me deep link with pre-filled amount
SEPA bank transfer integration	Completed	IBAN display with biometric guard and demo simulation
OCR receipt scanning	Partial	Google Cloud Vision API integrated; accuracy inconsistent on low-quality images; not promoted to main flow
Notifications system	Completed	Real-time Firestore listeners with Zustand store
Push notifications	Partial	Service worker registered; browser permission required; support varies
Activity feed	Completed	Chronological cross-group activity with filters

Analytics dashboard	Completed	Recharts spending trends, category breakdown, monthly overview
CSV and PDF export	Completed	Implemented following user feedback during evaluation
Dark mode	Completed	System preference detection with manual override
PWA installability	Completed	Android and iOS home screen install supported
Offline support	Partial	Shell and static assets cached; full data sync not yet implemented
Onboarding wizard	Completed	Action-first step-by-step guide for new users
In-app chat assistant	Completed	Firebase Cloud Function with Anthropic API
Unit and component testing	Completed	1,397 passing tests across 52 suites, 84.37% line coverage
End-to-end testing	Not Implemented	Cypress or Playwright deferred due to time constraint
Multi-language support	Not Implemented	English only; i18next integration planned as future enhancement

Native mobile application	Not Implemented	Outside scope; PWA approach adopted instead
Full banking API integration	Not Implemented	Outside scope; link-based payment model adopted

Table 1: Project Delivery checklist

Summary

Status	Count
Completed	29
Partial	3
Not Implemented	5
Total	**37**

Table 2: Project Delivery checklist Summary

Of the three partial features, OCR receipt scanning represents the most significant gap between intent and delivery and is discussed in detail in Sections 4.1 and 7.1. Offline data synchronisation and push notification reliability are constrained by browser API variability rather than

implementation gaps. Of the five not-implemented features, three were explicitly excluded from scope at project inception (end-to-end testing, native mobile application, and full banking API integration), while multi-language support and recurring expenses were deprioritised during the final development stage in favour of stability, testing coverage, and user-facing polish.

6. Problems Encountered and How They Were Resolved

Development rarely follows the planned path, and this project was no exception. What follows are specific problems I encountered, and the concrete steps taken to resolve them.

6.1 Firebase Security Rules Blocking Legitimate Reads

Problem: After deploying initial Firestore security rules, several features that had worked perfectly in development stopped functioning in the production environment. Group members could not read expense documents, and the invitation system failed silently.

Resolution: I used the Firebase Emulator Suite and the Firestore Rules Playground to simulate specific authenticated user requests against the rules. The issue was that the initial rules used request.auth.uid == resource.data.createdBy as the sole condition, which correctly blocked unauthenticated access but also blocked legitimate members who had not created the document. I rewrote the rules to check group membership using get() calls to retrieve the group document and verify that the requesting user's ID was present in the memberIds array. For example:

```
allow read: if request.auth != null && request.auth.uid in get(/databases/$(database)/documents/groups/$(groupId)).data.memberIds;
```

This resolved the issue but introduced a new concern around Firestore read costs, which I monitored through the Firebase console during testing.

6.2 React Testing Library and Firebase Mock Complexity

Problem: When I began writing component tests, the tests would crash immediately because components imported Firebase modules that tried to initialise the Firebase SDK, which is not available in the Jest test environment. Errors like 'Firebase App named [DEFAULT] already exists' and undefined function errors appeared constantly in early test runs.

Resolution: I created Jest manual mock files placed in a __mocks__ directory. The firebaseAuth.js mock exported stub functions for `signInWithEmailAndPassword`, `createUserWithEmailAndPassword`, `onAuthStateChanged`, and others. The firebaseFirestore.js mock stubbed `onSnapshot`, `getDocs`, `addDoc`, and related Firestore functions. Once these mocks were correctly wired up in setupTests.js using `jest.mock()` calls, the test environment became stable. The key lesson was that each mock needed to return the correct shape of data, not just an empty object, because components destructure returned values and would crash if the shape was wrong.

6.3 Balance Calculation Edge Cases

Problem: During early user testing, a scenario was discovered where a group of three people with multiple expenses produced negative balances that were mathematically correct but displayed confusingly in the UI. A user who had overpaid across multiple expenses was shown as owing money in one context and being owed money in another, which was technically accurate but unintuitive.

Resolution: I refactored the balanceCalculator.js to produce a single net balance per user pair rather than tracking individual expense debts. The settlement suggestion algorithm was then

updated to use a greedy approach: at each step, the person who owes the most pays the person who is owed the most, minimising the total number of transactions required to settle the group. This reduced a four-person settlement that previously required six transactions down to three in most test cases, and the display became much clearer.

6.4 OCR Receipt Scanning Accuracy

Problem: The Google Cloud Vision API integration worked technically, in that it could extract text from receipt images, but the accuracy for extracting the total amount was inconsistent. Irish supermarket receipts in particular often have multiple totals printed (subtotal, VAT breakdown, final total) and the parsing logic was sometimes selecting the wrong value.

Resolution: I implemented a filtering and scoring system that prioritised amounts appearing after keywords such as 'Total', 'Amount Due', 'Grand Total', and 'Balance Due' while discarding amounts preceded by 'Subtotal', 'VAT', or 'Discount'. I also added a confidence threshold so that if no high-confidence total was found, the user was prompted to enter the amount manually rather than receiving an incorrect pre-filled value. This improved accuracy for well-formatted receipts but the feature still does not handle all formats reliably, which is why it was not promoted to the main user flow.

6.5 Dark Mode Inconsistency Across Components

Problem: As the number of components grew, maintaining consistent dark mode styling became increasingly difficult. Several components that were built early in the project used hardcoded white backgrounds and dark text colours that did not respond to the DarkModeContext, resulting in jarring visual inconsistencies when toggling between modes.

Resolution: I audited every component systematically, replacing hardcoded colour values with Tailwind CSS conditional classes using the dark: variant prefix. For example, bg-white became bg-white dark:bg-gray-800, and text-gray-900 became text-gray-900 dark:text-white. The DarkModeContext was also extended to write the 'dark' class to the document root element so that Tailwind's dark mode class strategy worked correctly. This took a full day of work but resulted in fully consistent dark mode support across the application.

6.6 Session Timeout and Navigation Race Conditions

Problem: The session timeout hook (useSessionTimeout.js) was logging users out correctly but the navigation back to the login screen sometimes occurred before the Firebase signOut promise had resolved, leaving the user in an inconsistent state where the UI had changed but the auth token was still technically valid.

Resolution: I refactored the logout sequence to await the Firebase signOut call before triggering the React Router navigation. The auth state listener in the authStore Zustand store then handled clearing the user state reactively, so the component tree would update correctly once signOut completed. Adding an explicit loading state during the sign-out process also prevented any UI interaction during the brief window between initiating and completing sign-out.

7. What Was Not Achieved and Why

Being honest about what did not make it into the final deliverable is important, both academically and personally. The following features were planned but not delivered, along with the reasons.

7.1 Full OCR Receipt Scanning

As discussed in Section 4.1, the receipt scanning feature was implemented but not production ready. The primary reason was time. Getting the parsing logic to a level of reliability that would genuinely help rather than frustrate users would have required significantly more iteration and a wider dataset of receipt formats to test against. The decision was made to keep it as a visible but clearly beta feature rather than remove it entirely, as it demonstrates the technical integration even if the UX is imperfect.

7.2 End-to-End Testing

Cypress or Playwright end-to-end testing was on the roadmap from the beginning. The reason it was not implemented is straightforward: by the time the testing phase began, the remaining time before the submission deadline was insufficient to set up a full E2E test suite alongside the unit testing work. E2E testing with Firebase requires either a test Firebase project or the Firebase Emulator, and configuring this correctly within a CI-friendly setup would have taken at least a week of work that could not be prioritised.

7.3 Export Functionality (PDF and CSV)

The ability to export expense history as a PDF or CSV was planned as a quality-of-life feature. Libraries like `jsPDF` or the native browser print API would have made this achievable, but it was cut from scope during the final stages initially to focus on stability and testing coverage but after feedback from users, it was implemented.

7.4 Budget Tracking and Alerts

A budget tracking feature that would allow groups or individuals to set spending limits and receive alerts when approaching them was designed conceptually but not implemented. The data model supports it, as expense amounts and categories are already stored per user, but the UI and notification logic were not built. This was a prioritisation decision rather than a technical barrier.

7.5 Multi-Language Support

Internationalisation using a library such as i18next was not implemented. The application is English-only. Given the target market includes multiple EU countries where English is not the primary language, this is a genuine limitation. It was excluded because localisation work is time-intensive and the feature set itself was more important to complete first.

8. What I Would Do Differently if Starting Again

Reflection is a useful exercise, and there are several decisions I would reconsider if I were beginning this project from scratch.

8.1 Write Tests From Day One

The single biggest mistake I made was treating testing as a late-stage activity rather than an integral part of the development process. Having to retroactively write tests for 29 files and reach 50% coverage in a concentrated effort near the deadline was stressful and, frankly, less effective than test-driven development would have been. If I were starting again, I would write tests alongside every new feature, even if they were simple smoke tests initially. The balance calculator was one module where I did apply this approach, and it produced noticeably better code because I was forced to think about edge cases before writing the implementation.

8.2 Invest More Time in Data Modelling Upfront

Several refactors during development arose because the Firestore data model was not fully thought through before implementation began. The notifications collection structure went through two iterations. Firestore's NoSQL nature means that querying patterns need to be known before the model is designed, because you cannot join collections efficiently. A more thorough upfront design phase for the data model would have saved two or three days of refactoring work during development.

8.3 Use TypeScript

The entire project is written in JavaScript. There were several bugs during development that TypeScript would have caught at compile time: incorrect property names on objects returned from Firestore, mismatched function parameter types, and missing fields in mock objects during testing. The overhead of adding TypeScript to a React project is manageable, and for a project of this size and complexity, the type safety would have paid dividends. I would use TypeScript from the start if beginning again.

8.4 Scope the OCR Feature More Conservatively

The receipt scanning feature absorbed more development time than its current level of completeness justifies. I found OCR integration genuinely interesting and spent time exploring it that might have been better directed toward more user-facing features. If I were scoping the project again, I would either commit fully to making the OCR feature production-ready or exclude it entirely and use that time on end-to-end testing or additional UI polish.

8.5 Set Up CI/CD Earlier

While I deployed regularly to Firebase Hosting throughout development, I did not configure a proper continuous integration pipeline. A simple GitHub Actions workflow that ran the test suite on every push and blocked merges if tests failed would have caught several regressions earlier and established better development discipline. Setting this up takes perhaps half a day and the benefit across a multi-month project would have been significant.

8.6 Engage Supervisor More Regularly

My supervisor, Joseph Kehoe, was consistently helpful when I did meet with him, but I sometimes went longer than I should have between check-ins, usually because I wanted to resolve a problem myself first. In hindsight, some issues that took me several days to resolve might have been solved much faster with an earlier conversation. Regular weekly or bi-weekly meetings would have been a better cadence than the less frequent interactions that actually occurred.

9. Mobile vs Desktop comparison

Split-Sync was designed as a mobile-first PWA, and this is reflected in the feature comparison below. The majority of core features — expense splitting, balance tracking, group management, payments, and analytics — are fully functional across both mobile and desktop environments. However, several features are exclusive to, or significantly enhanced by, the mobile platform. This section provides a structured comparison of the features and capabilities of Split-Sync when used on a mobile device versus a desktop or laptop computer. The purpose is to inform development priorities, highlight platform-specific constraints, and support the evaluation of the application in the context of my Final Year Project.

Legend to note:

Legend	
Full Support	*Feature is fully functional and optimised*
Partial Support	*Feature works but with some limitations*
Limited	*Feature available but constrained by platform*

Mobile Only	*Feature exclusive to mobile devices*
Desktop Only	*Feature exclusive to desktop browsers*
Not Available	*Feature not supported on this platform*

Table 3: Mobile vs Desktop Legend

9.1 Core Expense Management

The following features form the primary functionality of Split-Sync and are available on both platforms.

Feature	Mobile	Desktop	Notes
Add Expense	**Full Support**	**Full Support**	*Fully functional on both platforms via the Add Expense modal.*
Equal Split	**Full Support**	**Full Support**	*Calculates and distributes expenses equally among all selected members.*

Percentage Split	**Full Support**	**Full Support**	*Custom percentages per participant; validation ensures totals equal 100%.*
Exact Amount Split	**Full Support**	**Full Support**	*Manual amount entry per participant with real-time total validation.*
Split by Shares	**Full Support**	**Full Support**	*Proportional share allocation (e.g., 2x, 1x) across all members.*
Expense Categories	**Full Support**	**Full Support**	*Categories include Food, Travel, Entertainment, Utilities, and more.*
Expense Description	**Full Support**	**Full Support**	*Optional free-text notes field for each expense.*
Multi-Currency Groups	**Full Support**	**Full Support**	*Each group can be assigned a different currency (GBP, EUR, USD, etc.).*
Edit Expense	**Full Support**	**Full Support**	*Expense details can be updated after creation by any group member.*

Delete Expense	**Full Support**	**Full Support**	*Expenses can be deleted with a confirmation prompt.*
Expense Date Selection	**Full Support**	**Full Support**	*Custom date can be set per expense; defaults to current date.*
Export CSV and PDF	**Full Support**	**Full Support**	*Export as Spreadsheet or browser pdf Print*

Table 4: Core Expense Management

9.2 Balance Tracking and Settlement

Feature	Mobile	Desktop	Notes
Real-Time Balance Tracking	**Full Support**	**Full Support**	*Balances update live via Firestore listeners on both platforms.*
Balance Breakdown Modal	**Full Support**	**Full Support**	*Drill-down view showing who owes whom across all groups.*
Settlement Summary	**Full Support**	**Full Support**	*Per-group settlement overview with minimum transaction optimisation.*
Individual Balances	**Full Support**	**Full Support**	*Per-member balance displayed within each group.*
Settle Up Modal	**Full Support**	**Full Support**	*Allows users to record a payment with amount confirmation.*

Paid Offline Recording	**Full Support**	**Full Support**	*Marks a settlement as paid outside the app (cash, bank transfer).*
Settlement Notifications	**Full Support**	**Full Support**	*Payer and payee both receive in-app notifications on settlement.*
Overall Net Balance	**Full Support**	**Full Support**	*Dashboard summary of total owed and total to receive across all groups.*

9.3 Payment Integrations

Feature	Mobile	Desktop	Notes
PayPal Payment Link	**Full Support**	**Full Support**	*Generates a PayPal.me deep link for the exact settlement amount.*
Revolut Payment Link	**Full Support**	**Full Support**	*Generates a Revolut payment link with pre-filled amount.*

SEPA Bank Transfer	**Full Support**	**Full Support**	*Displays payee IBAN, BIC, name and reference for manual transfer.*
SEPA QR Code	**Full Support**	**Partial Support**	*QR code generated for SEPA; scanning via camera is easier on mobile.*
Payment Link Deep Link	**Mobile Only**	**Partial Support**	*Deep links open PayPal/Revolut apps directly on mobile. Desktop opens browser.*
Payment Method Setup	**Full Support**	**Full Support**	*Users configure PayPal username, Revolut tag and IBAN in Profile settings.*
Biometric Payment Guard	**Mobile Only**	**Limited**	*WebAuthn biometric prompt available on mobile. Desktop relies on platform authenticator.*

Table 6: Payment Integrations Feature Comparison

9.4 Authentication and Security

Feature	Mobile	Desktop	Notes
Email and Password Login	**Full Support**	**Full Support**	*Standard Firebase email/password authentication on all platforms.*
Google OAuth Login	**Full Support**	**Full Support**	*One-tap Google sign-in supported on mobile and desktop.*
Biometric Authentication	**Full Support**	**Limited**	*Fingerprint and Face ID via WebAuthn. Desktop support depends on hardware (e.g., Windows Hello, Touch ID on Mac).*
Session Timeout Warning	**Full Support**	**Full Support**	*Inactivity timeout modal prompts re-authentication after a set period.*
Forgot Password Flow	**Full Support**	**Full Support**	*Password reset via Firebase email link on both platforms.*

End-to-End Encryption	**Full Support**	**Full Support**	*Sensitive payment data (IBAN) encrypted before storage using AES-GCM.*
Secure Payment View	**Full Support**	**Full Support**	*Payment settings hidden behind biometric or password re-authentication.*

Table 7: Authentication and Security Feature Comparison

9.5 Group Management

Feature	Mobile	Desktop	Notes
Create Group	**Full Support**	**Full Support**	*Group name, icon, currency and initial members set on creation.*
Group Icon Selector	**Full Support**	**Full Support**	*Icon picker with category-based icons (Home, Travel, Food, etc.).*

Invite via Link	**Full Support**	**Full Support**	*Shareable invite link generated and copyable on both platforms.*
Invite via QR Code	**Full Support**	**Partial Support**	*QR code can be scanned with mobile camera. Desktop requires a QR reader.*
Join Group via Link	**Full Support**	**Full Support**	*Invite link opens the Join Group page on both platforms.*
Member Management	**Full Support**	**Full Support**	*View all members with their email, photo, and SEPA-ready status.*
Quick Add Members	**Full Support**	**Full Support**	*Search existing Split-Sync users by name or email to add to a group.*
Contact Sync (Find Friends)	**Mobile Only**	**Not Available**	*Uses device contacts API to find friends already using Split-Sync. Not available on desktop browsers.*

Group Currency Setting	**Full Support**	**Full Support**	*Currency is set at group creation and displayed throughout the group.*

Table 8: Group Management Feature Comparison

9.6 Receipt Scanning (OCR)

Feature	Mobile	Desktop	Notes
Camera Capture	**Full Support**	**Not Available**	*Mobile camera directly captures a receipt photo within the app.*
File Upload Fallback	**Full Support**	**Partial Support**	*Image file can be uploaded on desktop as an alternative to camera.*
OCR Text Extraction	**Full Support**	**Partial Support**	*Google Cloud Vision API processes the image on both platforms; results quality depends on image quality.*

Auto-fill Expense Fields	**Full Support**	**Partial Support**	*Merchant name, total and date auto-populated from scan; requires good lighting on mobile.*
Receipt Preview	**Full Support**	**Partial Support**	*Scanned receipt image previewed before confirming on mobile. Desktop preview depends on file type.*

Table 9: Receipt Scanning (OCR) Feature Comparison

9.7 Notifications

Feature	Mobile	Desktop	Notes
In-App Notifications	**Full Support**	**Full Support**	*Bell icon with unread badge in the top navigation on all platforms.*
Notifications Dropdown	**Full Support**	**Full Support**	*Expandable dropdown listing all recent notifications.*

New Expense Notification	**Full Support**	**Full Support**	*Members notified when a new expense is added to their group.*
Settlement Notification	**Full Support**	**Full Support**	*Payee notified when a settlement payment is confirmed by payer.*
Invitation Notification	**Full Support**	**Full Support**	*Group invite notification with Accept and Decline actions.*
Push Notifications	**Partial Support**	**Partial Support**	*Service worker supports push; requires user permission grant. Browser support varies.*
Notification Badge Count	**Full Support**	**Full Support**	*Unread count badge displayed on the notification bell icon.*

Table 10: Notifications Feature Comparison

9.8 User Interface and Experience

Feature	Mobile	Desktop	Notes
Dark Mode	**Full Support**	**Full Support**	*Dark and light themes, respecting system preference or manual toggle.*
Bottom Navigation Bar	**Full Support**	**Not Available**	*Tab bar (Home, Activity, Add, Stats, You) shown on mobile only.*
Top Navigation Bar	**Partial Support**	**Full Support**	*Top bar with settings and notifications. Desktop layout has more horizontal space.*
Responsive Layout	**Full Support**	**Full Support**	*Tailwind CSS fluid grid adapts to all screen sizes.*
Keyboard Awareness	**Mobile Only**	**Not Available**	*useKeyboardAware hook scrolls inputs into view when the soft keyboard appears.*

Scrollable Modals	**Full Support**	**Full Support**	*All modals are max-height constrained with internal scroll on both platforms.*
Touch Gestures	**Full Support**	**Not Available**	*Touch-optimised tap targets and spacing. Not applicable on desktop.*
Hover States	**Not Available**	**Full Support**	*Hover effects on buttons, rows and links on desktop pointer devices.*
Onboarding Wizard	**Full Support**	**Full Support**	*First-login guided setup wizard displayed on both platforms.*
Tooltip Component	**Full Support**	**Full Support**	*Contextual tooltips available on hover (desktop) and tap (mobile).*

Table 11: User Interface and Experience Feature Comparison

9.9 Spending Analytics

Feature	Mobile	Desktop	Notes
Spending Charts	**Full Support**	**Full Support**	*Animated Recharts bar and pie charts for spending breakdown.*
Category Breakdown	**Full Support**	**Full Support**	*Expense totals grouped by category across all groups.*
Activity Feed	**Full Support**	**Full Support**	*Chronological list of all expense and settlement activity.*
Monthly Overview	**Full Support**	**Full Support**	*Total spending per month displayed across platforms.*
Chart Responsiveness	**Partial Support**	**Full Support**	*Charts render correctly on both platforms; wider desktop view offers more readable axis labels.*

Table 12: Spending Analytics Feature Comparison

9.10 Progressive Web App (PWA) Capabilities

Feature	Mobile	Desktop	Notes
Installable to Home Screen	**Full Support**	**Partial Support**	*Install prompt supported on Android Chrome. iOS Safari requires manual "Add to Home Screen". Desktop Chrome supports install.*
Standalone Display Mode	**Full Support**	**Partial Support**	*Runs fullscreen without browser chrome when installed on mobile. Desktop install opens in app window.*
Offline Support	**Full Support**	**Partial Support**	*Service worker caches shell. Offline page shown when network is unavailable. Full offline data sync not yet implemented.*
Service Worker	**Full Support**	**Full Support**	*Registered on both platforms; handles caching and offline fallback page.*

App Icon on Device	**Full Support**	**Partial Support**	*Maskable icons provided for Android and iOS home screens. Desktop shortcut icon set.*
Splash Screen	**Mobile Only**	**Not Available**	*Theme colour splash screen shown on mobile app launch. Not applicable to desktop.*

Table 13: Progressive Web App (PWA) Capabilities Feature Comparison

9.11 Settings and Profile Management

Feature	Mobile	Desktop	Notes
Display Name Update	**Full Support**	**Full Support**	*Name synced across Firebase Auth, Firestore user document and all group member arrays.*
Profile Photo Upload	**Full Support**	**Partial Support**	*Photo upload via camera (mobile) or file picker (desktop). Stored in Cloudinary.*

Payment Methods Setup	**Full Support**	**Full Support**	*PayPal, Revolut and SEPA IBAN configurable in Profile settings on both platforms.*
Biometric Toggle	**Full Support**	**Limited**	*Enable or disable WebAuthn biometric in Settings. Hardware-dependent on desktop.*
Session Timeout Config	**Full Support**	**Full Support**	*Inactivity timeout configurable in Settings.*
Account Deletion	**Full Support**	**Full Support**	*Permanent account deletion with confirmation flow on both platforms.*
Logout	**Full Support**	**Full Support**	*One-tap logout available from Profile and Settings pages.*
Dark Mode Toggle	**Full Support**	**Full Support**	*Manual override for light/dark mode available in Settings.*

Table 14: Settings and Profile Management Feature Comparison

Mobile Advantages

- Direct camera access for receipt scanning (OCR auto-fill).
- Contact synchronisation to discover friends using Split-Sync via phone contacts.
- Full biometric authentication using fingerprint or Face ID via WebAuthn.
- Keyboard-aware scrolling that repositions inputs when the soft keyboard appears.
- Payment deep links that open PayPal and Revolut native apps directly.
- Full PWA install experience with splash screen and standalone mode on Android and iOS.
- Touch-optimised UI with bottom navigation bar and appropriate tap target sizing.

Desktop Advantages

- Wider screen real estate allows more data to be visible simultaneously in tables and charts.
- Hover states and keyboard navigation improve productivity for power users.
- Analytics charts display with more readable axis labels and spacing on wider viewports.
- File-based workflows (uploading receipts, downloading exports) are more natural on desktop.

Platform Constraints

- Contact Sync is not available on desktop browsers due to the absence of the Contacts API.
- Camera-based receipt scanning is not available on desktop; file upload is the fallback.
- Biometric authentication on desktop depends on the availability of platform authenticators (Windows Hello, Touch ID).
- PWA install experience on iOS requires manual steps and lacks the native install banner.

Overall, Split-Sync delivers a consistent and functional experience across both platforms, with mobile offering a richer native-feeling experience due to hardware integrations, and desktop providing a more spacious and pointer-friendly interface for data management tasks.

10. User Evaluation

10.1 Methodology

A structured online questionnaire was designed and distributed to evaluate the usability, feature satisfaction, and overall user experience of Split-Sync. The form was hosted at splitsync-feedback.netlify.app and built using Formspree as the submission backend. The questionnaire comprised seven sections covering participant demographics, overall experience, feature ratings, interface and performance, improvement suggestions, comparative analysis, and recommendation intent. Responses were collected between 22 March 2026 and 6 April 2026. A total of 18 valid responses were received from participants with varied use cases including flatmates, couples, dining groups, and travel groups.

10.2 Participant Overview

Participants were recruited through direct sharing of the feedback link among personal contacts and app testers. The primary use cases reported were dining and social outings (39%, n=7), flatmates and housemates (28%, n=5), and shared expenses with a partner (28%, n=5), with travel expenses accounting for one participant (6%, n=1). Usage frequency was predominantly a few times a month (56%) and a few times a week (33%), with one participant reporting rare usage.

10.3 Overall Experience

Overall Rating: The mean overall rating across all 18 participants was **4.67 out of 5**. Fifteen participants awarded a rating of 5, two awarded 4, and one awarded 3. This indicates a strongly positive reception of the application.

Ease of Use: The mean ease-of-use rating was **4.44 out of 5**. Fourteen participants rated ease of use as 5 (Very Easy), three as 4 (Easy), and one as 3 (Neutral). This demonstrates that the majority of users found the application intuitive without significant onboarding friction.

Expectations: Nine participants (50%) reported that Split-Sync exceeded their expectations, eight (44%) reported their expectations were met, and one (6%) reported expectations were mostly met. No participant reported that the application fell short of expectations.

10.4 Feature Ratings

Participants were asked to rate specific features on a scale of 1 to 5, with the option to select N/A for features not used. The mean scores for each feature are presented below.

Feature	Mean Rating	Participants Rating
Balance Tracking	5.00 / 5	17
Expense Splitting	4.82 / 5	17
Biometric Authentication	4.93 / 5	14
Receipt Scanning	4.62 / 5	13
Payment Links	4.27 / 5	15

Table 15: User Evaluation Feature Ratings by Participants

Balance tracking and biometric authentication received the highest satisfaction scores. Payment links received a slightly lower average, which is consistent with qualitative feedback requesting additional payment methods such as Google Pay and Apple Pay.

Most Valuable Feature Expense splitting was identified as the most valuable feature by 11 participants (61%), followed by analytics by 3 participants (17%), and balance tracking by 2 participants (11%).

10.5 Interface and Performance

Visual Design: The mean design rating was **4.72 out of 5**, indicating strong satisfaction with the visual presentation of the application.

Performance: The mean performance rating was **4.61 out of 5**, reflecting that the application was perceived as fast and responsive by the majority of users.

Mobile Experience: The mean mobile experience rating was **4.78 out of 5**, confirming that the Progressive Web App delivered a high-quality experience on mobile devices.

Bugs and Errors: Fourteen participants (78%) reported no bugs. Two participants (11%) encountered minor issues that did not significantly affect their experience. One participant reported significant bugs affecting usability, specifically noting difficulty understanding the payment confirmation flow. One participant noted a missing welcome screen during authentication on first use.

10.6 Qualitative Findings

Open-ended responses were grouped into recurring themes through thematic analysis.

Theme 1 — Export and Reporting: Multiple participants expressed a desire to export group expenditure data as a PDF or CSV file. This was the most consistently requested functional addition, appearing across five responses.

Theme 2 — Automated Payments and Settlement: Two participants noted the need for more automation in the payment confirmation process. Specifically, the requirement to manually mark payments as received was identified as friction in the settlement flow. One participant noted that having to confirm with their bank before marking a payment was an inconvenience.

Theme 3 — Additional Payment Methods: Three participants requested support for Google Pay and Apple Pay. One participant also requested more currency options and language localisation beyond English.

Theme 4 — Recurring and Scheduled Expenses: Recurring expenses was the single most requested feature addition, appearing across eleven responses. Participants who split regular costs such as rent or subscriptions found the absence of this feature notable.

Theme 5 — App Store Availability: Two participants asked whether Split-Sync would be available on the Google Play Store or Apple App Store, reflecting genuine intent to use the application as a primary expense management tool beyond the academic context.

Theme 6 — Notifications and Push Alerts: Several participants requested improved push notifications and the ability to receive alerts directly on their device rather than only within the app.

Selected Participant Quotes

One participant wrote: "If this wasn't a school project, it would have been my go-to app. It is very nice."

Another noted: "Payments with Google Pay and Apple Pay will be nice to have. I wish it will be available in the app store."

A third commented: "I wanted the place where I could see all my friends I have connected with or those using Split-Sync."

10.7 Comparative Analysis

Eleven participants (61%) had previously used alternative expense-splitting applications including Splitwise, Settle Up, and manual tracking via spreadsheets or notes. Of those who had used alternatives, eleven (100%) rated Split-Sync as much better than the alternatives they had tried. One participant rated it as somewhat better and one rated it as about the same. Five participants had no prior experience with similar applications and could not make a comparison.

10.8 Net Promoter Score

Participants were asked how likely they would be to recommend Split-Sync to a friend or colleague on a scale of 0 to 10. Based on standard NPS methodology, scores of 9 and 10 are classified as Promoters, 7 and 8 as Passives, and 0 to 6 as Detractors (Reichheld, 2003).

Category	Count	Percentage
Promoters (9–10)	14	77.8%
Passives (7–8)	3	16.7%
Detractors (0–6)	1	5.6%

Table 16: Net Promoter Score Classification

NPS = 77.8% − 5.6% = 72.2

An NPS of 72 is considered excellent and falls within the range associated with industry-leading consumer applications (Bain and Company, 2023). This result demonstrates strong user advocacy for the application.

Retention Intent: Eight participants (44%) stated they would definitely continue using Split-Sync. Nine (50%) said they probably would, and one (6%) was unsure. No participant stated they would not continue using the application.

10.9 Observational and Informal User Testing

In addition to the structured questionnaire, informal observational testing was conducted with several individuals who used the application directly during the evaluation period. These sessions were not formally recorded but provided valuable qualitative insight that complemented the survey findings.

10.9. 1 Cybersecurity Peer Review

The application was shared with colleagues from a cybersecurity background who attempted to identify vulnerabilities through common attack vectors. Their testing included attempting direct URL manipulation to access restricted group pages, injecting malicious input into form fields including expense titles, group names, and payment fields, and inspecting application behaviour through the browser developer console and network tab. No unauthorised data access was achieved. Firestore security rules correctly blocked unauthenticated and unauthorised read and write attempts. This informal penetration exercise prompted a review of all input fields across the application, resulting in the addition of strict maxLength constraints and server-side format validation on all payment inputs including IBAN (maximum 34 characters), BIC (maximum 11 characters with regex enforcement), PayPal username (maximum 20 characters), and Revolut username (maximum 30 characters). The review confirmed that the API key for Anthropic is secured server-side within Firebase Cloud Functions and is never exposed to the client.

10.9.2 General App Testers

Several additional participants used the application informally without completing the feedback form. Key observations from these sessions included the following. One user attempted to add an expense immediately after creating a group before any other member had joined, causing confusion about why balances were not appearing. This led to the implementation of the disabled Add Expense button and amber warning banner for solo groups. Another user noted that when they changed their display name in Settings, it was not reflected in group member lists until the page was refreshed, which led to the fix of updating the Zustand authentication store

immediately on name change so all components reflect the new name in real time. A third user navigating between multiple groups with different currencies found that the Profile dashboard was displaying a single combined total with a hardcoded euro symbol, which was misleading. This observation was the primary trigger for the full multi-currency refactor across Profile, Activity, and Analytics. One user also noted that the group icon display in the Quick Add Expense screen was showing the icon key name as plain text rather than the actual icon, which was corrected by implementing the Lucide icon map with distinct colours per group type.

10.10 Reflection and Response to Feedback

Several pieces of feedback from both the structured questionnaire and observational testing were directly addressed during the development cycle prior to submission. The absence of a clear invite-before-expense flow was addressed by disabling the Add Expense button for solo groups and adding an amber prompt directing users to invite members first. The confusion around currencies across multiple groups was resolved by implementing a per-currency balance display across the Profile, Activity, and Analytics pages, ensuring amounts are never incorrectly summed across different currencies. Input validation was strengthened across all payment fields following security feedback. The onboarding wizard was redesigned to follow an action-first structure, clearly differentiating required setup steps from optional features. Video guides were also made available in the Support Section of the application settings. CSV and PDF export has been included since users requested it.

Some features requested by participants that could be added as Premium features for business and were not implemented within the project timeline — including recurring expenses, Google Pay and Apple Pay integration, and language localisation — are acknowledged as meaningful future enhancements. These represent a clear and validated product roadmap grounded in real user need.

11. Conclusion

Split-Sync represents the most substantial piece of software I have built independently to date. It encompasses technologies and challenges that touch on almost every area covered during the BSc programme: database design, authentication, API integration, state management, security, testing, and deployment. The fact that it is a live, deployed, functional application that I use myself gives me a genuine sense of achievement.

The project fell short in some areas, most notably the production-readiness of the OCR feature, the absence of end-to-end testing, and the uneven distribution of test coverage. These are not failures I am trying to minimize. They are honest outcomes of the constraints of solo development within a fixed academic timeline, and understanding why they happened is itself a valuable learning outcome.

What I am most proud of is the testing milestone. Going from 1.32% coverage to 84.37% in a systematic, methodical way, building a proper mocking infrastructure and writing 1,397 meaningful tests across 52 test suites, was the part of this project that felt most like professional software engineering practice rather than student project work.

Split-Sync has a clear path forward. The codebase is clean, well-structured, and ready to be extended. The features that were not completed within the academic timeline are documented and designed; they simply need time to implement. If I continue developing the application after submission, the foundation is solid enough to support that.

This project has made me a significantly more capable developer than I was when it began, and that, ultimately, is what a Final Year Project is supposed to do.

Glossary

Term	Definition
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode. A symmetric encryption algorithm used in Split-Sync to encrypt sensitive payment data, including IBAN numbers, before storage in Firestore ISO (2007).
API (Application Programming Interface)	A set of defined protocols and tools that allows different software systems to communicate (PayPal Developer, 2024) & (MDN, 2024). Split-Sync integrates with APIs including Google Cloud Vision, PayPal, Revolut, Firebase, and Anthropic Claude.
Biometric Authentication	A security mechanism that verifies a user's identity using physical characteristics such as a fingerprint or facial recognition. Split-Sync implements biometric authentication via the WebAuthn API for payment confirmation Firebase (2024).
BIC (Bank Identifier Code)	An eight or eleven character code used internationally to identify banks in financial transactions. Also referred to as a SWIFT code. Required alongside IBAN for SEPA transfers.

Cloud Function	A serverless computing unit provided by Firebase (Google Cloud, 2024). Split-Sync uses a Cloud Function to host the in-app AI chat assistant, ensuring the Anthropic API key is never exposed to the client.
DarkModeContext	A React context provider used in Split-Sync to manage the application-wide dark mode state. It detects system preferences and supports manual toggle, writing the 'dark' class to the document root element.
Firestore	A NoSQL cloud database provided by Firebase (Google LLC). Split-Sync uses Firestore to store user accounts, group data, expense records, notifications, and invitation documents in real time.
GDPR (General Data Protection Regulation)	A European Union regulation (EU) 2016/679 governing data privacy and protection. Split-Sync's contact sync feature uses SHA-256 hashing to align with GDPR principles by avoiding server-side storage of raw contact data.
Google Cloud Vision API	A machine learning image analysis API provided by Google. Split-Sync uses it within the ReceiptScanner component to perform Optical Character Recognition (OCR) on uploaded receipt images.

IBAN (International Bank Account Number)	A standardised international notation for identifying bank accounts across national borders. Used in Split-Sync's SEPA payment flow; validated using a modulo-97 checksum algorithm across over 30 European country formats.
Jest	A JavaScript testing framework developed by Meta formerly Facebook (Jest (2024) and React Testing Library (2024). Used throughout Split-Sync for unit and component testing. All 1,397 tests are written using Jest with React Testing Library.
Lucide React	An open-source icon library for React applications (Lucide, 2024). Used throughout Split-Sync in place of emojis to maintain a consistent and professional visual language.
NPS (Net Promoter Score)	A customer experience metric that measures user willingness to recommend a product, calculated as the percentage of Promoters (score 9-10) minus the percentage of Detractors (score 0-6). Split-Sync achieved an NPS of 72.2 in user evaluation.
OAuth 2.0	An open authorisation framework that enables users to grant limited access to their accounts without sharing passwords. Split-Sync supports Google OAuth 2.0 for authentication via Firebase Authentication.

OCR (Optical Character Recognition)	The process of extracting text from images. In Split-Sync, OCR is used in the ReceiptScanner component to auto-fill expense fields (merchant name, total, and date) from uploaded receipt images via the Google Cloud Vision API.
Progressive Web App (PWA)	A web application that uses modern browser capabilities to deliver an app-like experience (W3C, 2021). Split-Sync is a PWA supporting installability on Android and iOS home screens, offline fallback pages, and service worker caching.
React	An open-source JavaScript library for building user interfaces, developed by Meta. Split-Sync is built using React 18, utilising functional components, hooks, and a context-based state management pattern.
React Testing Library	A testing utility for React that encourages testing components from the user's perspective. Used in combination with Jest to write component tests throughout Split-Sync.
Recharts	A composable charting library built on React and D3 (Recharts, 2024). Used in Split-Sync's Analytics page to render animated bar charts, pie charts, and monthly overview graphs for spending data.

SEPA (Single Euro Payments Area)	A European payment integration initiative enabling standardised euro-denominated bank transfers across 36 countries (EPC, 2023 & EU, 2016). Split-Sync implements SEPA bank transfer support including IBAN display, BIC, and a Demo Mode simulation.
Service Worker	A background script registered in the browser that enables PWA functionality such as offline caching and push notification delivery. Split-Sync's service worker caches the application shell and static assets.
SHA-256 (Secure Hash Algorithm 256-bit)	A cryptographic one-way hash function from the SHA-2 family standardised by NIST (2015). Used in Split-Sync's contact synchronisation feature to hash phone numbers locally before any comparison occurs, preventing raw contact data from leaving the device.
Tailwind CSS	A utility-first CSS framework used for styling throughout Split-Sync (Tailwind CSS, 2024). The dark: variant prefix is used to apply conditional styles in dark mode, and the fluid grid system supports responsive layouts across mobile and desktop.
WebAuthn (Web Authentication API)	A W3C standard that enables strong authentication using public-key cryptography in the browser. Split-Sync uses WebAuthn to support fingerprint and Face ID authentication for payment confirmation and profile access.

Zustand	A lightweight state management library for React. Split-Sync uses Zustand stores for authentication state (authStore), notification state (notificationsStore), and other global UI state, replacing the need for a heavier solution such as Redux.

Appendix

Appendix A: Project Links and Deployed Artefacts

A.1 Live Application

Split-Sync is deployed at: https://splitsync-dev.web.app

The application is publicly accessible and may be used to verify the features described throughout this report.

A.2 Source Code Repository

The full source code is hosted on GitHub at: https://github.com/EmmanuelAbolade/Split-sync.git

All commits are chronologically organised and reflect the iterative development approach described in Section 5.

A.3 Test Coverage Report

The interactive Jest coverage report is hosted at: https://splitsync-coverage-test.netlify.app/

This report was generated using the Jest --coverage flag and presents statement, branch, function, and line coverage per file.

A.4 User Feedback Form and Application Showcase

The user evaluation questionnaire was hosted at: https://splitsync-feedback.netlify.app

A feature showcase page is available at: https://splitsync-showcase.netlify.app/

A.5 Screencast Video

A screencast walkthrough of the application is available at: https://youtu.be/7uat5PeTsrI?si=0_pGz91OvCN4-UQ6

The video demonstrates core user journeys including group creation, expense entry, balance settlement, and payment integration.

A.6 Doxygen Technical Documentation

Auto-generated code documentation is hosted at: https://splitsync-docs.netlify.app/

Appendix B: Test Coverage Summary Table

The table below summarises the overall test coverage position at submission, as reported by the Jest coverage runner. Full per-file detail is available via the link in Appendix A.3.

Metric	Statements	Branches	Functions	Lines
Coverage at Final Submission	82.14%	71.23%	80.11%	84.37%
Target (Milestone)	80%	70%	80%	80%
Tests Passing	1,397	—	—	—
Test Suites	52	—	—	—

Appendix C: User Evaluation Questionnaire Structure

The structured evaluation form contained the following seven sections, distributed via Formspree (form ID: mqeyogqd) and hosted on Netlify. Responses were collected between 22 March 2026 and 6 April 2026.

#	Section	Content
1	Demographics	Primary use case (flatmates, dining, travel, partner), usage frequency.
2	Overall Experience	Overall rating (1-5), ease of use (1-5), whether expectations were met.
3	Feature Ratings	Ratings for: Balance Tracking, Expense Splitting, Biometric Auth, Receipt Scanning, Payment Links.
4	Interface and Perf.	Visual design rating, performance rating, mobile experience rating, bug reports.
5	Improvements	Open-ended: most valuable feature, suggested additions, most requested feature.
6	Comparative Analysis	Prior use of alternatives (Splitwise, Settle Up), comparative rating.
7	Recommendation	Net Promoter Score (0-10), retention intent, willingness to recommend.

Appendix D: Firebase Security Rules (Simplified Extract)

The following is a simplified extract of the Firestore security rules used in production. Full rules are available in firestore.rules within the source repository.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
        match /groups/{groupId} {
        allow read: if request.auth != null &&
        request.auth.uid in resource.data.memberIds;
        allow write: if request.auth != null &&
        request.auth.uid == resource.data.createdBy;
        }
  }
}
```

Appendix E: IBAN Validation Algorithm

Split-Sync implements client-side IBAN validation using the modulo-97 checksum algorithm as defined by ISO 13616 and the European Payments Council (2023). The algorithm proceeds as follows:

Step 1: Remove spaces and convert the IBAN to uppercase.

Step 2: Move the first four characters to the end of the string.

Step 3: Replace each letter with its numeric equivalent (A=10, B=11, ... Z=35).

Step 4: Compute the remainder of dividing the resulting integer by 97.

Step 5: The IBAN is valid if and only if the remainder equals 1.

This logic is implemented in sepaPayment.js and covers over 30 European country formats.

Appendix F: Settlement Algorithm Description

The settlement suggestion algorithm in balanceCalculator.js implements a greedy two-pointer approach to minimise the number of transactions required to settle a group. The algorithm operates as follows:

Step 1: Compute the net balance for each group member (total paid minus total owed).

Step 2: Separate members into two lists: those with a positive net balance (creditors) and those with a negative net balance (debtors).

Step 3: Sort both lists by absolute value in descending order.

Step 4: Pair the largest debtor with the largest creditor. The smaller absolute value determines the transaction amount.

Step 5: Subtract the transaction amount from both parties. Remove any member whose balance reaches zero.

Step 6: Repeat until all balances are zero.

This algorithm produces the minimum number of transactions required to settle the group, and was validated through the balanceCalculator.js test suite with 100% statement coverage.

Bibliography

Abramov, D. and Clark, A. (2015) Redux: A predictable state container for JavaScript apps. Available at: https://redux.js.org (Accessed: 10 November 2025).

Anthropic (2024) Claude AI assistant. Available at: https://claude.ai (Accessed: 11 March 2026).

Aßmann, S., Borning, A. and Dahl, O.-J. (2010) 'Debt-settlement algorithms in group financial management', ACM SIGPLAN Notices, 45(10), pp. 23–31.

Auth0 (2024) Introduction to JSON Web Tokens. Available at: https://jwt.io/introduction (Accessed: 2 December 2025).

Bain and Company (2023) The Net Promoter System. Available at: https://www.netpromotersystem.com (Accessed: 6 April 2026).

Cloudinary (2024) Cloudinary image and video management documentation. Available at: https://cloudinary.com/documentation (Accessed: 15 November 2025).

Cohn, M. (2004) User Stories Applied: For Agile Software Development. Boston: Addison-Wesley Professional.

European Payments Council (2023) SEPA Credit Transfer Scheme Rulebook. Brussels: EPC. Available at: https://www.europeanpaymentscouncil.eu (Accessed: 8 January 2026).

European Union (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). Official Journal of the European Union, L 119, pp. 1–88. Available at: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679 (Accessed: 20 November 2025).

FIDO Alliance (2022) Web Authentication (WebAuthn) specification. Available at: https://fidoalliance.org/fido2/fido2-web-authentication-webauthn (Accessed: 20 November 2025).

Firebase (2024) Firebase documentation: Firestore, Authentication, Hosting, Storage. Google LLC. Available at: https://firebase.google.com/docs (Accessed: 12 February 2026).

Freeman, S. and Pryce, N. (2009) Growing Object-Oriented Software, Guided by Tests. Boston: Addison-Wesley Professional.

Google Cloud (2024) Cloud Vision API documentation. Available at: https://cloud.google.com/vision/docs (Accessed: 5 December 2025).

ISO (2007) ISO 13616-1:2007: Financial services — International bank account number (IBAN) — Part 1: Structure of the IBAN. Geneva: International Organization for Standardization.

Jest (2024) Jest JavaScript testing framework documentation. Available at: https://jestjs.io/docs (Accessed: throughout testing phase, 2025–2026).

Kent C. Dodds (2021) Common mistakes with React Testing Library. Available at: https://kentcdodds.com/blog/common-mistakes-with-react-testing-library (Accessed: 18 January 2026).

Lucide (2024) Lucide React icon library. Available at: https://lucide.dev (Accessed: 1 December 2025).

Martin, R.C. (2009) Clean Code: A Handbook of Agile Software Craftsmanship. Boston: Prentice Hall.

Mozilla Developer Network (2024) Web APIs: Service Workers API, WebAuthn API, IndexedDB API. Available at: https://developer.mozilla.org/en-US/docs/Web/API (Accessed: 15 February 2026).

National Institute of Standards and Technology (2015) FIPS PUB 180-4: Secure Hash Standard (SHS). Gaithersburg: NIST. Available at: https://csrc.nist.gov/publications/detail/fips/180/4/final (Accessed: 22 November 2025).

Nielsen, J. (1994) Usability Engineering. San Diego: Academic Press.

PayPal Developer (2024) PayPal payment links and deep link documentation. Available at: https://developer.paypal.com/docs (Accessed: 12 December 2025).

React (2024) React 18 documentation. Meta Open Source. Available at: https://react.dev (Accessed: throughout development, 2025–2026).

React Testing Library (2024) Testing Library documentation. Available at: https://testing-library.com/docs/react-testing-library/intro (Accessed: 4 December 2025).

Recharts (2024) Recharts: A composable charting library for React. Available at: https://recharts.org/en-US (Accessed: 3 February 2026).

Reichheld, F.F. (2003) 'The one number you need to grow', Harvard Business Review, 81(12), pp. 46–54.

Revolut (2024) Revolut payment links documentation. Available at: https://developer.revolut.com (Accessed: 12 December 2025).

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.

Tailwind CSS (2024) Tailwind CSS documentation v3.4. Available at: https://tailwindcss.com/docs (Accessed: 27 February 2026).

W3C (2021) Progressive Web Apps: Definition and core criteria. World Wide Web Consortium. Available at: https://www.w3.org/TR/appmanifest (Accessed: 15 October 2025).

W3C (2021) Web Authentication: An API for accessing Public Key Credentials Level 2. World Wide Web Consortium. Available at: https://www.w3.org/TR/webauthn-2 (Accessed: 20 November 2025).

Zustand (2024) Zustand state management library documentation. Available at: https://zustand-demo.pmnd.rs (Accessed: 20 November 2025).